



The Recorder That Never Lets You Forget a Promise

A Dustin OS white paper — how a Plaud recorder, a local transcriber, and one nightly pass turned "I'll get back to you on that" into something that actually gets tracked. Educational reference only.

Published by The Appraiser Coach. Educational reference — not a build guide, not a product release. Promised to the Appraiser AI Architects room on September 13, 2026; finally getting it out the door.

↓ Download as PDF

Formatted PDF — for sharing, archival, or posting to a community forum.

Read this first — disclaimer

- **This is a white paper, not a software release.** No source code is published here. There is no installer and no "download and run" path.
- **No warranty, no tech support.** If you build something like this, you're on your own for troubleshooting.
- **Nothing here is legal or compliance advice.** If you're a covered business under state or federal privacy law, get your own read on call-recording and AI-processing consent requirements before you touch any of this. Plaud's own terms of service govern what you may do with its device and app; read those too.
- **None of the actual conversations described here are shared or quoted.** This paper describes the plumbing, not the content that flows through it.

Contents

1. The problem: promises made out loud, kept nowhere
2. Why "no upload" was the actual requirement
3. Layer 1 — the recorder never asks you to do anything
4. Layer 2 — one nightly pass finds the promises
5. How it reaches you: no new app to check
6. If you want to build your own version
7. What we tried and dropped

1. The problem: promises made out loud, kept nowhere

A working appraiser has more conversations than emails. A borrower calls about timing. An agent catches you at the property and asks for a rush. A lender asks you to look into a comp before Friday. Every one of those calls carries a small commitment, and none of them land in an inbox, a CRM, or a to-do list on their own. You either write it down in the moment — unlikely, you're driving or mid-inspection — or you trust yourself to remember it. Most of the time you do remember. It's the one time you don't that costs you a client.

The obvious fix is "record everything and review the recordings." In practice, nobody does that. Reviewing your own calls is exactly the kind of task that feels important and never survives contact with a full calendar. A system that requires you to sit down and listen to yesterday's calls is a system that will be abandoned within two weeks. Whatever solved this had to require nothing from me after the conversation ended.

2. Why "no upload" was the actual requirement

A Plaud recorder (or any wearable/desk recorder with a companion app) already solves the capture half of this: press record, talk, it syncs to the cloud. Most people's workflow stops there — the recording sits in an app, transcribed or not, and reviewing it is still a task you have to remember to do. The step that actually matters is the one nobody wants to do by hand: pulling each new recording out, running it through something that understands it, and deciding whether anything in it needs following up on.

So the design constraint wasn't "transcribe my calls." It was: **nothing about this should require me to open an app, tap export, or upload a file anywhere.** If a system depends on a human remembering to feed it, it will eventually go unfed, and the whole point was to stop depending on memory in the first place.

3. Layer 1 — the recorder never asks you to do anything

The device syncs recordings to the cloud on its own, the way it's designed to. The part that's custom is a small watcher that runs on a schedule (hourly, in this build) and checks for anything new since the last pass. When it finds a recording:

1. It pulls the audio down programmatically — no phone, no export button, no manual step.

2. It converts it to a plain transcript. Note: not by uploading the audio to a third-party AI transcription API. It runs entirely on the local machine using an open-source speech-to-text engine ([whisper.cpp](https://github.com/openai/whisper), an implementation of OpenAI's Whisper model that runs offline with no per-minute cost). That choice came out of a real failure: an earlier version of this used a cloud transcription API, and it silently ate a two-hour workshop recording mid-run when the API account ran out of prepaid credit. A system meant to make you trust it less by hand cannot have a failure mode where it quietly loses a recording. Running locally means there's no bill that can run out, no vendor outage that takes down the pipeline, and no length limit on the recording.
3. The finished transcript gets filed with the date and the source recording's ID, and it's marked as handled so the same file is never reprocessed.

At the end of this layer, every conversation you've recorded exists as searchable text, without you ever touching it. That alone is useful — "what did she actually say about the closing date" becomes a text search instead of a re-listen. But it's still just an archive. The second layer is what makes it a second brain instead of a filing cabinet.

4. Layer 2 — one nightly pass finds the promises

Every morning, before anything else runs, a second job looks at everything new from the day before: fresh transcripts, plus (in this build) the emails you personally sent and any working sessions you had with your own AI assistant. It asks one specific question about all of it: **did I commit to doing or deciding something here — and did anyone commit something to me that I'm now waiting on?**

That question matters more than it looks. A naive version of this would flag every sentence that sounds like a task and bury you in noise — "let's circle back," "I'll think about it," small talk that isn't a real commitment. The useful version is narrow on purpose:

- **It only looks for genuine commitments** — something a reasonable person would call you out for forgetting, not idle conversation.
- **It checks what's already being tracked** and drops anything that's a near-duplicate of something already sitting there, so the same promise doesn't pile up night after night.
- **It caps how much lands per day.** A handful of the most important, longest-overdue items, not a wall of text. The point is a five-minute glance, not a new inbox to dread.
- **Anything that goes unresolved for too long ages out on its own** into an archive rather than nagging forever. If it was actually important, it'll come up again.
- **If the whole pass fails for any reason, it says so loudly** rather than quietly doing nothing. A tracker that fails silently is worse than no tracker, because you stop checking it yourself once you trust it.

5. How it reaches you: no new app to check

This is the part that makes the whole thing sustainable: the output doesn't go anywhere new. It lands in the same place every other open item in this system already lives, which already feeds the daily morning brief you were reading anyway. There is no second dashboard, no separate to-do app, no notification channel to configure and then ignore. If you already have a "here's what's on your plate today" system, the fix isn't to build a parallel one for recorded promises — it's to make sure recorded promises feed the one you already check.

The underlying principle, if you take nothing else from this paper: automation that requires you to remember to use it will fail exactly at the moments you need it most — when you're busy. Every step in this pipeline, from the recorder syncing itself to the transcript filing itself to the promise landing in a place you already look, was chosen because it needed nothing from me in the moment.

6. If you want to build your own version

This is an outline, not a build guide. At minimum, you'd need:

- **A recording device with programmatic access to its own recordings** — an API, an official connector, or an export you can reach without a person tapping a button each time. Plaud has one; other recorder apps increasingly do too. Read the terms of service for what you're allowed to do with your own recordings before you build anything against it.
- **A local transcription step you control.** whisper.cpp is free, offline, and has no cliff to fall off when a prepaid balance hits zero. A cloud transcription API works too, but budget for what happens when it's down or your account runs dry mid-recording — that failure should degrade gracefully, not silently lose data.
- **A scheduled job, not a manual one.** Whatever runs the "find the promises" pass has to run on its own, on a timer, with no human trigger. The moment it needs you to kick it off, you're back to the original problem.
- **A tightly scoped prompt for the extraction step** that distinguishes "a real commitment" from "a sentence that sounds like one." This is the part worth iterating on the most — too loose and you get noise you'll start ignoring; too strict and you miss the thing that actually mattered.
- **Guardrails against runaway noise:** a cap on how much surfaces per day, deduplication against what's already tracked, and an age-out so nothing nags forever.
- **One existing place to land in.** Whatever you already check every morning — that's where this should write to. Don't build a second inbox.

7. What we tried and dropped

Approach	Why we tried it	Why we dropped it
Cloud transcription API	Fast to stand up, no local compute needed, decent accuracy out of the box.	A metered API account ran out of prepaid credit mid-transcription and silently lost most of a long workshop recording. A system built to reduce how much you have to remember cannot itself have a failure mode where it quietly forgets. Moved to a local, offline model with no balance to run out.
Flagging every task-shaped sentence	Simple to build — just pattern-match on "I will," "let's," "can you."	Buried real commitments under small talk and hedged language. A tracker nobody trusts gets ignored, which defeats the entire point.
A separate dashboard for recorded-conversation follow-ups	Felt like it deserved its own view, since it's a distinct source.	A second place to check is a second place to forget to check. Folded into the one existing daily summary instead.
Reviewing recordings manually on a schedule ("Friday catch-up")	The obvious starting point — no engineering required.	Never survived a busy week. The whole project exists because this didn't work.

8. Take-aways for the appraiser community

- **The device solves capture; it doesn't solve follow-through.** A recorder that syncs to an app is only half the job. The value is in what happens automatically after the recording exists, not in the recording itself.
- **"No upload" is a design constraint, not a nice-to-have.** Any workflow that needs a human to remember a manual step will eventually not get that step. Build backward from "what does this require of me after I hang up the phone" and aim for zero.
- **Run transcription somewhere it can't run out on you.** If you depend on a metered API for something you care about not losing, plan for the day the balance hits zero — or don't depend on a metered API for it.
- **Narrow the question you ask an AI to answer.** "Find anything task-shaped" produces noise. "Find a genuine commitment, made by me or to me" produces something you'll actually read.

- **Land it somewhere you already look.** The best task system is the one you're not tempted to ignore, and that's whichever one you're already checking every day.
- **None of this replaces judgment about what actually matters.** This surfaces candidates for your attention. Deciding what to do about each one is still entirely on you.

Repeat of the disclaimer

- This document describes a pattern, not a product. There is no software to download here, no installer, no source code.
- If you build something based on this, you're responsible for your own compliance with recording-consent law in your state, Plaud's (or any recorder's) terms of service, and any AI vendor's terms of service you use.
- Nothing in this document is legal advice. Talk to an attorney about call-recording consent requirements in your state before recording anyone else's side of a conversation.